Take-Home Final Examination

Questions marked (Hard) *may require a little extra reading slightly beyond the course material, or involve topics that were not heavily emphasized in lectures. Completing 3/4 of the questions correctly is enough for full score.*

1. Lambda Calculus. *(15 points)* *Lectures 1–3*

(a) Reduce to normal form, showing each step:

$$(\lambda f.\, \lambda x.\, f\ (f\ x))\ (\lambda y.\, \lambda z.\, z)$$

(b) Define a λ-term AND such that $\mathsf{AND}\ b_1\ b_2$ returns $\mathsf{TRUE} = \lambda x.\, \lambda y.\, x$ when both arguments are TRUE and $\mathsf{FALSE} = \lambda x.\, \lambda y.\, y$ otherwise. Verify one case by explicit reduction.

(c) The term $(\lambda x.\, y)\ ((\lambda z.\, z\ z)\ (\lambda z.\, z\ z))$ can be reduced in two ways. Describe both. Does the Church–Rosser theorem guarantee they reach the same result? Explain carefully.

2. STLC and Curry–Howard. *(15 points)* *Lectures 4–6*

(a) Give a typing derivation for $\lambda(f{:}\alpha \to \beta).\, \lambda(g{:}\beta \to \gamma).\, \lambda(x{:}\alpha).\, g\ (f\ x)$ in the empty context. What logical proposition does this type correspond to?

(b) Explain why $\lambda(x{:}A).\, x\ x$ is untypable in the STLC for any type A.

(c) Under the extended Curry–Howard correspondence (with $A \times B$ for $\wedge$ and $A + B$ for $\vee$), write a proof term for

$$A \wedge (B \vee C) \to (A \wedge B) \vee (A \wedge C).$$

Is $A \to \neg\neg A$ provable intuitionistically? (Recall $\neg A = A \to \bot$.) Either give a term or explain why not.

3. Normalization and System T. *(15 points)* *Lectures 9–10*

(a) (Hard) State what it means for a term to be weakly normalizing. Why can't the proof of weak normalization for the STLC proceed by induction on the typing derivation? What goes wrong in the application case?

(b) (Hard) System T extends the STLC with Nat, $\mathsf{0}$, S, and a recursor R_A with rules $\mathsf{R}\ z\ s\ \mathsf{0} \to_\beta z$ and $\mathsf{R}\ z\ s\ (\mathsf{S}\ n) \to_\beta s\ n\ (\mathsf{R}\ z\ s\ n)$. Define add and mult using $\mathsf{R}_{\mathsf{Nat}}$. Verify $\mathsf{add}\ (\mathsf{S}\ \mathsf{0})\ (\mathsf{S}\ \mathsf{0}) \twoheadrightarrow_\beta \mathsf{S}\ (\mathsf{S}\ \mathsf{0})$.

(c) (Hard) What class of functions does System T capture? Give an example of a computable function not definable in System T.

4. System F. *(10 points)* *Lectures 11–12*

(a) In System F, the polymorphic boolean type is $\mathsf{Bool}_F = \forall\alpha.\, \alpha \to \alpha \to \alpha$. Write the terms for true and false. Define $\mathsf{NOT}_F : \mathsf{Bool}_F \to \mathsf{Bool}_F$ and verify by reduction that $\mathsf{NOT}_F\ \mathsf{true} = \mathsf{false}$.

(b) The type $\forall\alpha.\, \alpha \to \alpha$ is inhabited by exactly one closed term (up to $\beta\eta$-equivalence). What is it, and why must any inhabitant be the identity? (This is a free theorem.)

5. Monads. *(10 points)* *Lecture 13*

(a) Explain what problem `bind : m α → (α → m β) → m β` solves. Why can't we chain monadic computations with ordinary function composition?

(b) (Hard) Desugar the following `do`-block into explicit `bind`/`pure` calls, and trace the evaluation using the `Option` monad:

```
do let x ← some 5; let y ← none; pure (x + y)
```

(c) (Hard) Write a Lean function using the `State Nat` monad (with `get` and `set`) that increments the state three times and returns the final value. What does `yourFunction.run 0` evaluate to?

6. The λ-Cube and Dependent Types. *(15 points)* *Lectures 14–15*

(a) The λ-cube has eight systems determined by three axes: terms depending on types, types depending on types, and types depending on terms. Name the system at each corner that we studied: STLC (no axes), System F (one axis), and the Calculus of Constructions (all three). For each, give one example of a term or type that the system can express but the previous one cannot.

(b) In the system λP (types depending on terms), we can write $\mathsf{Vec} : \mathsf{Nat} \to \mathsf{Type}$ for length-indexed vectors. Give a type signature for a function append that concatenates $\mathsf{Vec}\ n$ and $\mathsf{Vec}\ m$ and returns $\mathsf{Vec}\ (n + m)$. Why can't this type be expressed in System F?

(c) (Hard) Girard's paradox shows that $\mathsf{Type} : \mathsf{Type}$ leads to inconsistency. What is the solution adopted by Lean? How does the universe hierarchy $\mathsf{Type}_0 : \mathsf{Type}_1 : \mathsf{Type}_2 : \ldots$ avoid the problem?

7. CIC and Inductive Types. *(15 points)* *Lectures 16–18*

(a) In the Calculus of Inductive Constructions, Prop and Type are both sorts but have different rules. What is the key difference? What is proof irrelevance, and why does it matter?

(b) (Hard) Given the inductive type Nat with constructors $\mathsf{zero} : \mathsf{Nat}$ and $\mathsf{succ} : \mathsf{Nat} \to \mathsf{Nat}$, state the type of the recursor $\mathsf{Nat.rec}$ that CIC generates. Explain what each argument of the recursor does.

(c) Lean distinguishes between *large elimination* and *small elimination* for types in Prop. When is large elimination allowed? Give an example of a Prop-valued inductive type that qualifies and one that does not, with a brief explanation of why.

8. Verified Programming in Lean. *(25 points)* *Lectures 7–8, 19–21*

(a) Define an inductive type `MyList (α : Type)` with constructors `nil` and `cons`. Define `myAppend : MyList α → MyList α → MyList α` and prove:

```
theorem append_nil (xs : MyList α) :
    myAppend xs nil = xs
```

(b) Define `myLength : MyList α → Nat` and prove:

```
theorem append_length (xs ys : MyList α) :
  myLength (myAppend xs ys) = myLength xs + myLength ys
```

(c) In Lectures 7–8 we saw two approaches to verified BSTs: extrinsic and intrinsic. Explain in a few sentences what each approach is. Give one situation where extrinsic verification is preferable to intrinsic, and one where intrinsic is preferable.

9. Type Classes and Well-Founded Recursion. *(10 points)* *Lectures 22–23*

(a) Define a type class `Describable` with a single method `describe : α → String`. Write instances for `Nat` and `Bool`. Write a polymorphic function `greet [Describable α] (x : α) : String` that returns `"Hello, " ++ describe x`. All code should compile.

(b) Define a function `log2 : Nat → Nat` that computes $\lfloor \log_2 n \rfloor$ by repeated halving ($\mathsf{log2}\ n = 1 + \mathsf{log2}\ (n/2)$ when $n > 1$, and 0 otherwise). Supply `termination_by`. Briefly explain: what does `termination_by` tell Lean, and what happens if you remove it?

10. Axioms and Computation. *(15 points)* *Lecture 24*

(a) Lean's kernel accepts three axioms beyond CIC: `propext`, `Quot.sound`, and `Classical.choice`. For each, state what it asserts (one sentence). Which of the three can block `#reduce` but are compatible with `#eval`? Which one requires `noncomputable`?

(b) The existential $\exists\, x : \alpha,\ P\ x$ carries a witness constructively. Explain why you nonetheless cannot write `def extract (h : ∃ n : Nat, n > 5) : Nat := h.1`. What rule of CIC prevents it? How does `Classical.choose` bypass this restriction?

(c) Diaconescu's theorem derives the law of excluded middle from `propext`, `funext`, and `Classical.choice`. The proof defines two sets U and V and picks elements $u = \text{Choice}\ U$, $v = \text{Choice}\ V$. Explain in your own words: why does $p \to u = v$ hold? (Your answer should mention what happens to U and V when p is true and why `Choice` gives the same result for both.)

Total: 145 points.